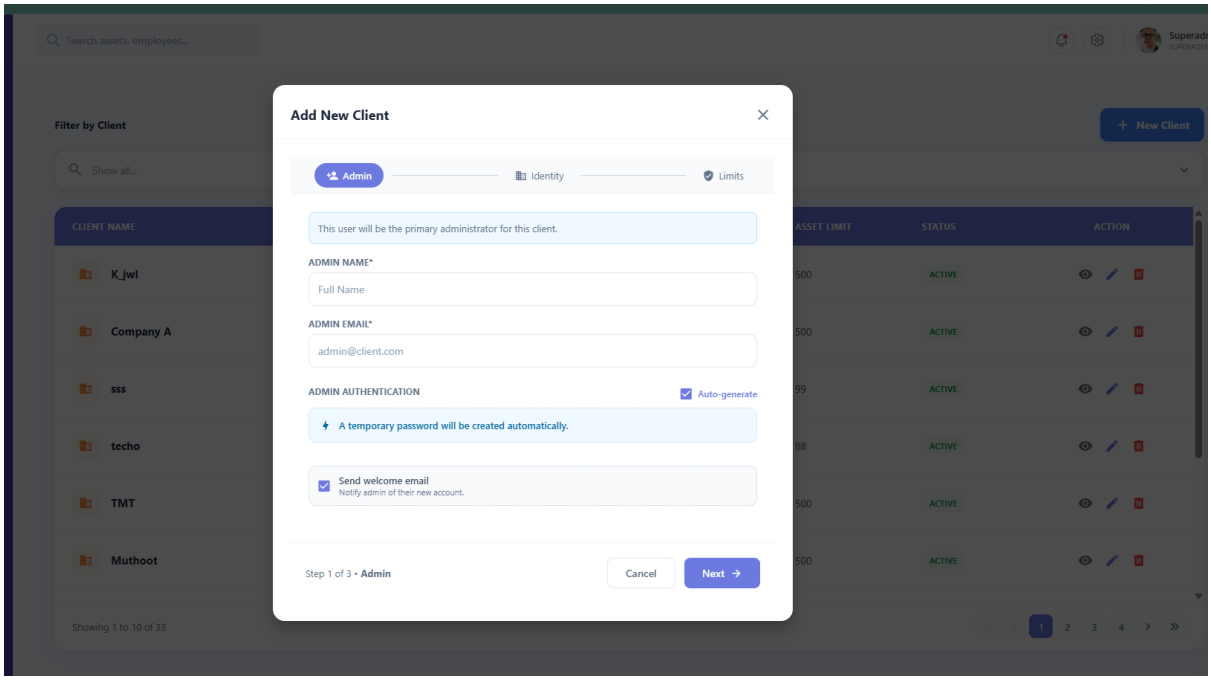
Asset Management System (Resoursix)

Resoursix is a centralized enterprise platform designed to manage and track the complete lifecycle of company assets, including equipment, vehicles, and premises, across multiple companies and branches. Built on a secure multi-tenant architecture, it ensures strict data isolation while allowing administrators to customize forms and workflows dynamically without hardcoding. The system provides full visibility from asset registration and assignment to maintenance and disposal, supported by real-time dashboards that help organizations improve control, reduce costs, and make data-driven operational decisions.

1.Clients Module – Technical Structure



1. Frontend (UI Layer)

File Path:
apps/web/src/screens/ClientsScreen.js

Purpose:

- Main screen for **Super Admin**
- Used to:

- Create new companies (clients)
- Edit company details
- Activate / deactivate companies
- Manage module access (Asset, Vehicle, Premises, etc.)

---

2. Backend (API Layer)

Controller

File:
server/controllers/clients.js

Purpose:

- Handles all business logic for clients:
  - Create company
  - Fetch company list
  - Update company details
  - Change status (Active / Inactive)
  - Delete company

Routes

File:
server/routes/clients.js

Purpose:

- Defines API endpoints such as:
  - GET /api/clients → Get all companies
  - POST /api/clients → Create new company
  - PUT /api/clients/:id → Update company
  - DELETE /api/clients/:id → Remove company
- Connects frontend requests → controller functions

---

3. Database (Schema Layer)

Main Table

Table Name: companies

Purpose:
This is the **core table** of the system.

It stores:

- company_id (Primary Key)
- Company name
- Email / contact info
- Login credentials (or linked auth)
- Status (Active / Inactive)
- Subdomain / unique identifier

Related Tables

1. users

- Stores all users (admins + employees)
- Each user is linked using company_id
- Ensures users belong to the correct company

---

2. company_modules

- Stores which modules are enabled for each company
- Example:
    - Asset → Enabled
    - Vehicle → Disabled
- Controls what the client can see after login

---

4. Multi-Tenant Logic (Simple Explanation)

- Every table includes a **company_id**
- This ensures:
    - Data is **separated per company**
    - No company can access another company's data

👉 Example:

- Company A → sees only its assets
- Company B → sees only its assets

---

5. Where to Find in Code

All structure is defined in:

Database File:
db/postgresql_schema.sql

You can quickly open files using:

- Ctrl + P → type:
    - ClientsScreen.js
    - clients.js
    - postgresql_schema.sql

---

6. Simple System Flow

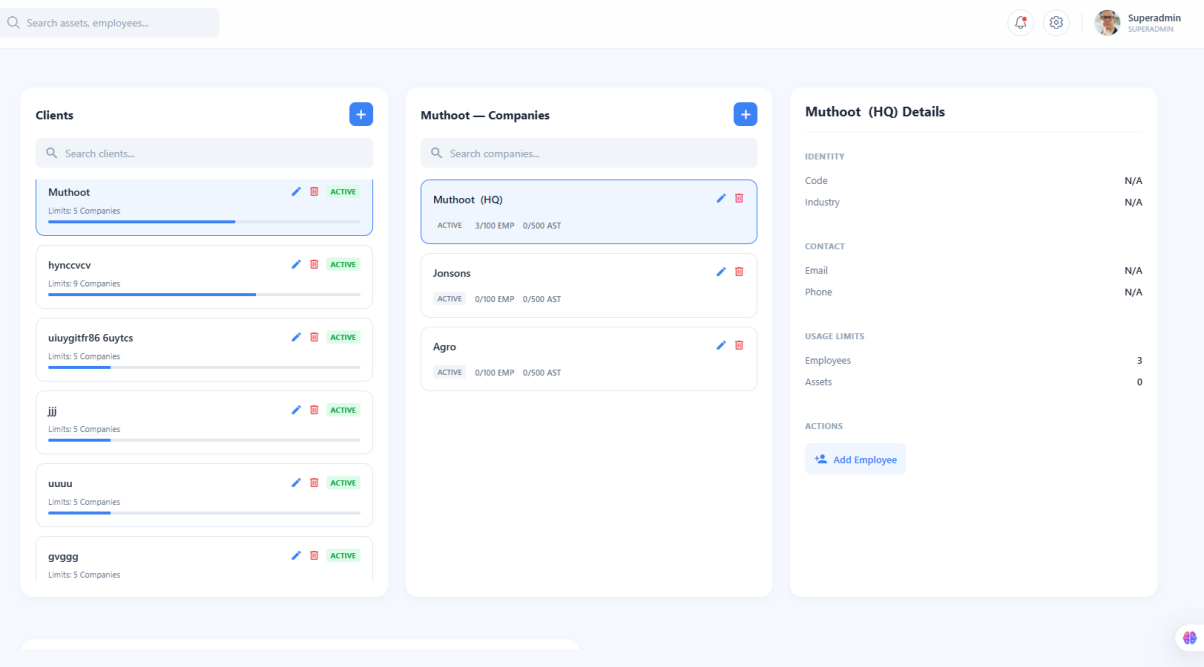
Frontend (ClientsScreen)
→ calls API

Routes (clients.js)
→ sends request to controller

Controller (clients.js)
→ processes logic

Database (companies + related tables)
→ stores and retrieves data

2.Company-Employee Connection



1. Hierarchy Structure (Data Isolation)

Level 1 – Main Client Group

- Example: **Muthoot**
- Acts as the **parent company**
- Can have multiple branches/companies under it
- Has a limit (e.g., **5 companies**)

Level 2 – Branches / Workspaces

- Example under Muthoot:
    - Muthoot (HQ)
    - Jonsons
    - Agro

How It Works

- Each branch works **independently**
- Data is separated using **company_id / branch_id**

👉 Example:

- Agro admin → sees only Agro data
- Jonsons admin → sees only Jonsons data

✓ No data is shared between branches

---

2. Add Employee Workflow

Location

- Right panel (selected branch details, e.g., *Muthoot (HQ)*)

Steps

1. **Select the Branch**
    - Click on a branch in the center panel
    - Example: *Muthoot (HQ)*, *Jonsons*, *Agro*
2. **Verify Selection**
    - Right panel updates (e.g., *Jonsons Details*)
3. **Click Add Employee**
    - Click **+ Add Employee**
    - Opens employee creation form
4. **Enter Employee Details**
    - Name
    - Email
    - Role
5. **Save Employee**
    - Employee is assigned to the selected branch

---

3. Backend & Database Mapping

Frontend (UI Layer)

File:
apps/web/src/screens/ClientsScreen.js

- Handles:
    - Branch selection
    - Add Employee button
    - Employee creation form

Backend (API Layer)

Controller:
server/controllers/employees.js *(or users.js)*

Routes:
server/routes/users.js *(or employees.js)*

Endpoints:

- POST /api/users
- POST /api/employees

Database (Schema Layer)

1. users

Stores login details:

- id
- company_id
- email
- password
- role_id

---

2. employees

Stores profile details:

- id
- user_id
- first_name
- last_name
- phone
- position

---

4. System Behavior (Data Isolation Logic)

- When adding an employee to a branch (e.g., **Jonsons**):

👉 System saves:
company_id / branch_id = Jonsons

Access Control

- Employee can access only:
    - Their assigned branch data
    - Allowed modules
- Employee cannot access:
    - Other branches (Muthoot HQ, Agro, etc.)

✓ No cross-branch data access

---

5. Key Rule

👉 **Each employee belongs to one branch only**

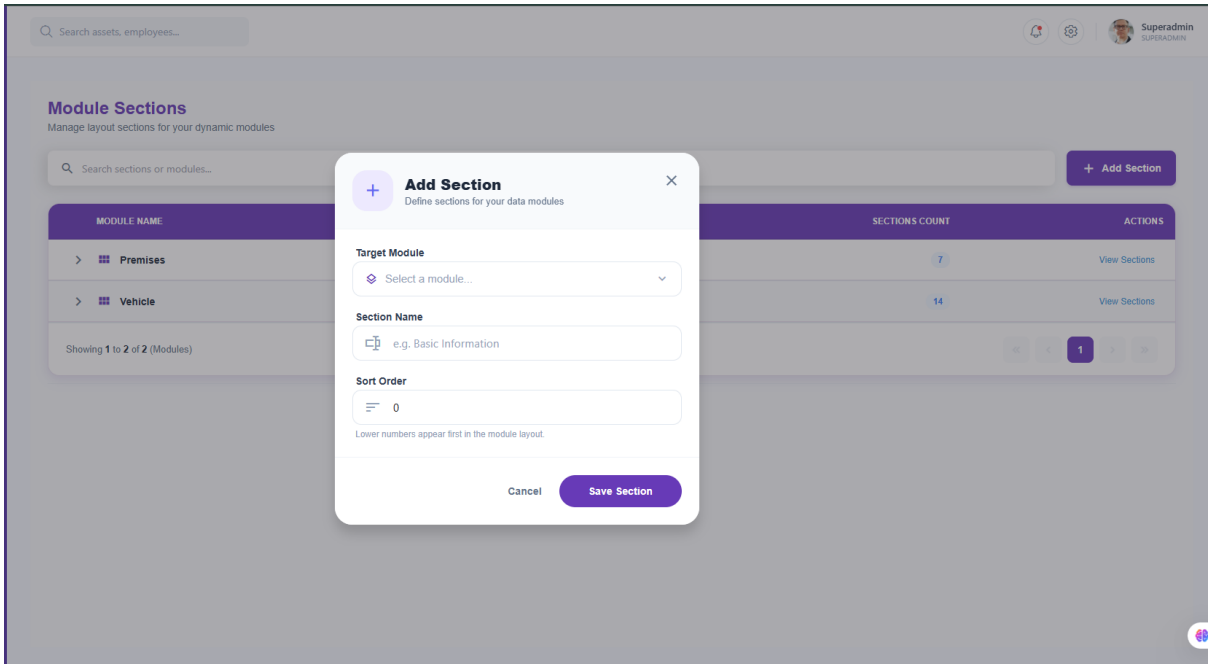
- Data is fully isolated
- Access is strictly controlled

---

6. Purpose

- Maintain **data isolation between branches**
- Control access based on company structure
- Ensure **secure multi-company operations**

3.Module Sections (Dynamic Headers) – Technical Structure



1. Frontend (UI Layer)

File Path:
apps/web/src/screens/modules/ModuleSectionsScreen.js

Purpose:
This screen is used to manage **Sections (Sub-modules / Headers)**.

Used to:

- Create new sections (e.g., *Identification*, *Specifications*)
- Assign sections to a module (Premises, Vehicles)
- Arrange sections using **sort order**
- Organize form layout into logical groups

---

2. Backend (API Layer)

Controller

File:
server/controllers/moduleSections.js

Purpose:

- Handles all operations for sections:
    - Create section
    - Get section list
    - Update section
    - Delete section

- Validates and saves section data

Routes

File:
server/routes/moduleSections.js

Purpose:

- Defines API endpoints:
    - GET /api/module-sections → Get all sections
    - POST /api/module-sections → Create section
    - PUT /api/module-sections/:id → Update section
    - DELETE /api/module-sections/:id → Delete section
- Connects frontend actions → controller

---

3. Database (Schema Layer)

Main Table

Table Name: module_sections

Stores:

- id
- company_id
- module_id
- name (Section Title)
- sort_order

Related Tables

1. modules

- Parent module (Premises, Vehicles)
- Each section belongs to one module

---

2. module_section_fields

- Stores fields under each section
- Links sections → actual input fields

---

4. System Flow (Simple)

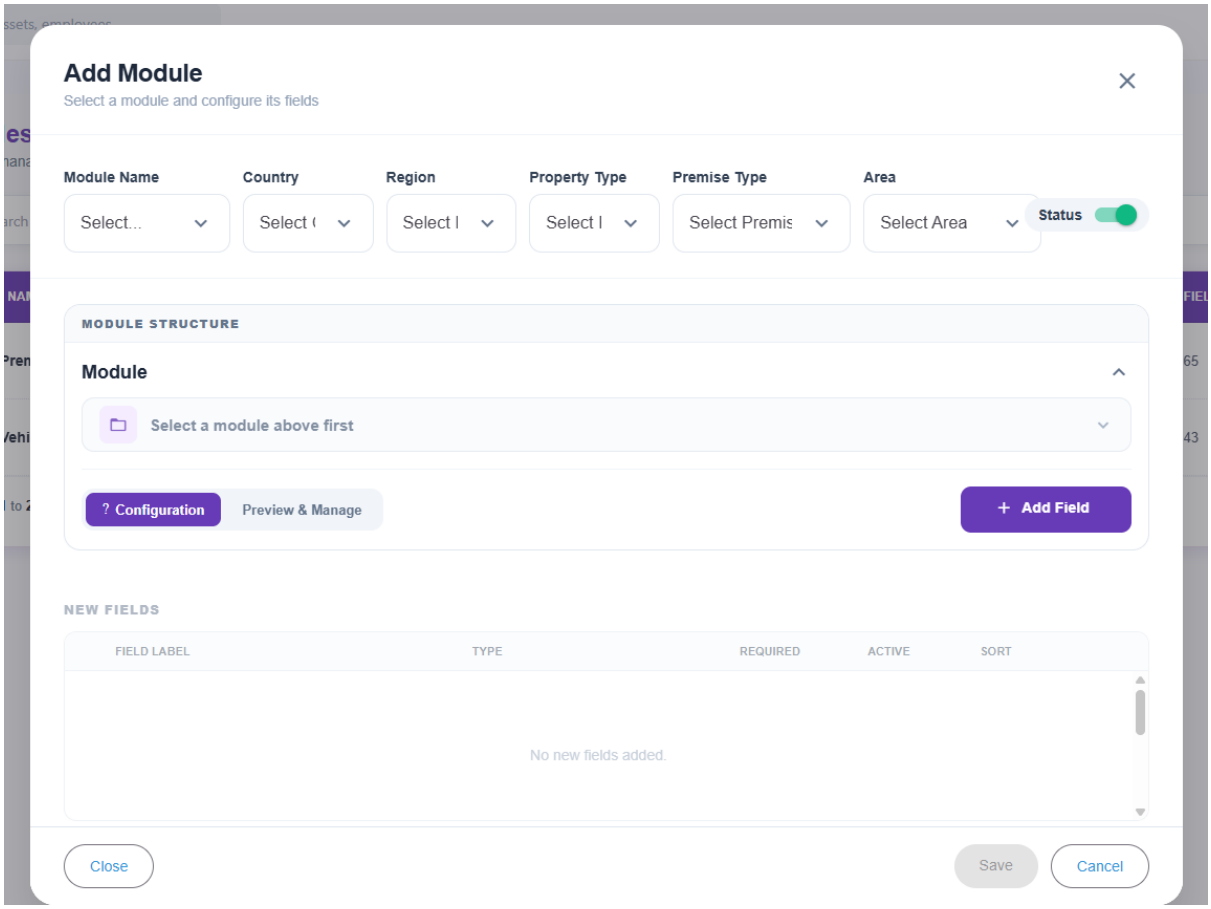
1. User opens **Module Sections screen**
2. User enters section name (e.g., *Detailed Specifications*)
3. User selects module (Premises / Vehicles)
4. User clicks **Save**
5. Frontend calls API → POST /api/module-sections
6. Controller validates and saves data
7. Section is stored in database
8. Section appears in UI and is used in form layout

---

5. Simple Purpose

- Organize fields into sections
- Improve form structure and readability
- Group related information together
- Control display order in forms

4. Dynamic Module Builder – Overview



This screen is used to **create custom forms and fields** inside modules like:

- Premises
- Vehicles

It allows admins to control **what fields appear and where they appear**.

---

1. Target Filters (Top Section)

This section controls **where the fields will be applied**.

- **Module Name** → Select module (Vehicle, Premises, etc.)
- **Country / Region / Area** → Apply fields to specific location
- **Property Type / Premise Type** → Example: only for *Rental* or *Owned*
- **Status** → Active or Draft

👉 Fields will appear only in selected conditions

---

2. Module Structure (Middle Section)

This section is used to **build and manage the form layout**.

- Select a module first
- **Configuration Tab** → Add and manage fields
- **Preview Tab** → See how form will look
- **+ Add Field** → Create new field (Text, Number, Dropdown, File, etc.)

---

3. Fields Preview (Bottom Section)

- Shows all created fields
- Displays:
    - Field Name
    - Input Type
    - Required / Optional
    - Order

👉 Controls final form structure

---

4. Code Mapping (Where It Exists in Code)

- **Frontend Screen:**
  apps/web/src/screens/modules/ModulesHomeScreen.js

👉 Handles:

- Modal (Add Module)
- Field creation
- Preview layout

---

5. Who Can Use This Module

✅ **Company Admin**

- Main user
- Can:
    - Create fields
    - Customize forms
    - Apply filters

✅ **Super Admin**

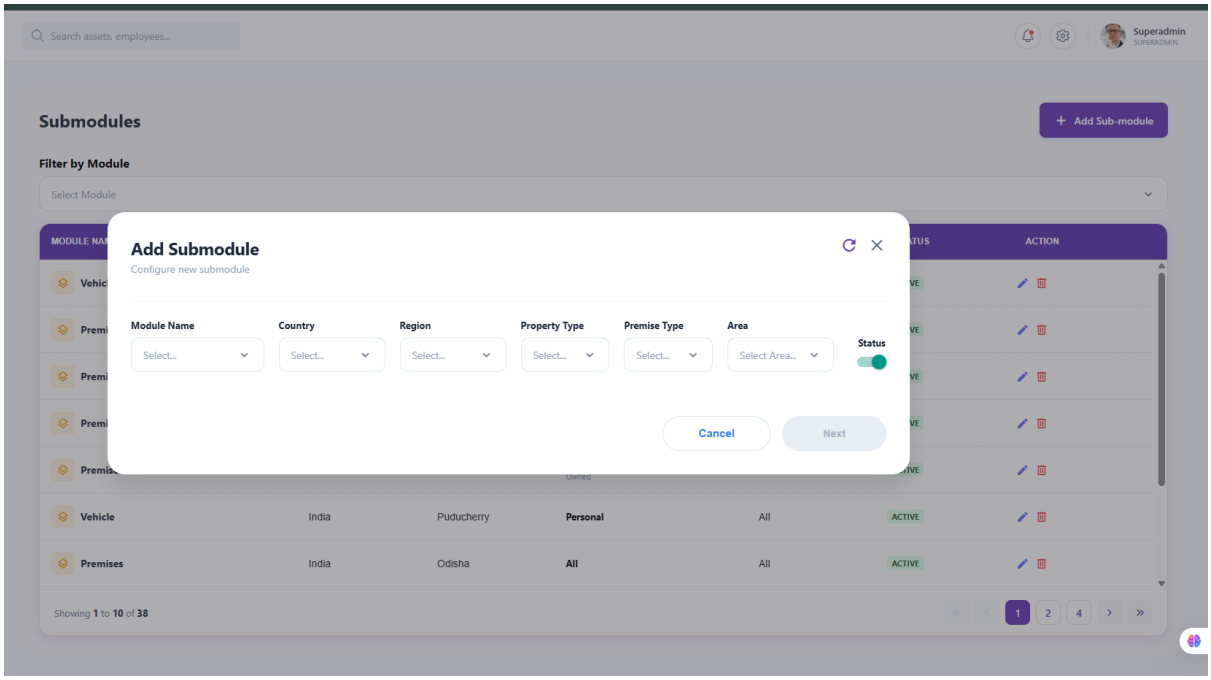
- Full access
- Can manage all configurations

---

6. Purpose

- Customize forms
- Control data collection
- Apply logic based on location or type
- Adapt system to company needs

5.Submodules – Technical Structure



1. Frontend (UI Layer)

File Path:
apps/web/src/screens/modules/SubModulesScreen.js

Purpose:
This screen is used to create **targeted configurations (Submodules)** for main modules.

Used to:

- Create sub-configurations for modules (Premises, Vehicles)
- Apply conditions based on location or type
- Customize how a module behaves in specific cases

Example:

- Vehicle module only for **Owned vehicles in India**

---

2. Backend (API Layer)

Controller

File:
server/controllers/companyModulesController.js

Purpose:

- Handles logic for submodule configurations
- Saves where and when a module configuration should apply
- Manages filtering conditions

Routes

File:
server/routes/companyModules.js

Purpose:

- Defines API endpoints for submodules:
    - Create configuration
    - Fetch configurations
    - Update configuration
    - Delete configuration
- Connects frontend → controller

---

3. Database (Schema Layer)

Main Table

Table Name: company_modules

Stores:

- company_id
- module_id
- country_id
- area_id
- premises_type_id (or other filters)

---

4. How It Works (Simple Logic)

- If **no filters are set** → module works for **all cases (global)**
- If **filters are set** → module works **only for selected conditions**

👉 Example:

- Country = India → applies only in India
- Premise Type = Rental → applies only for rental

---

5. System Flow (Simple)

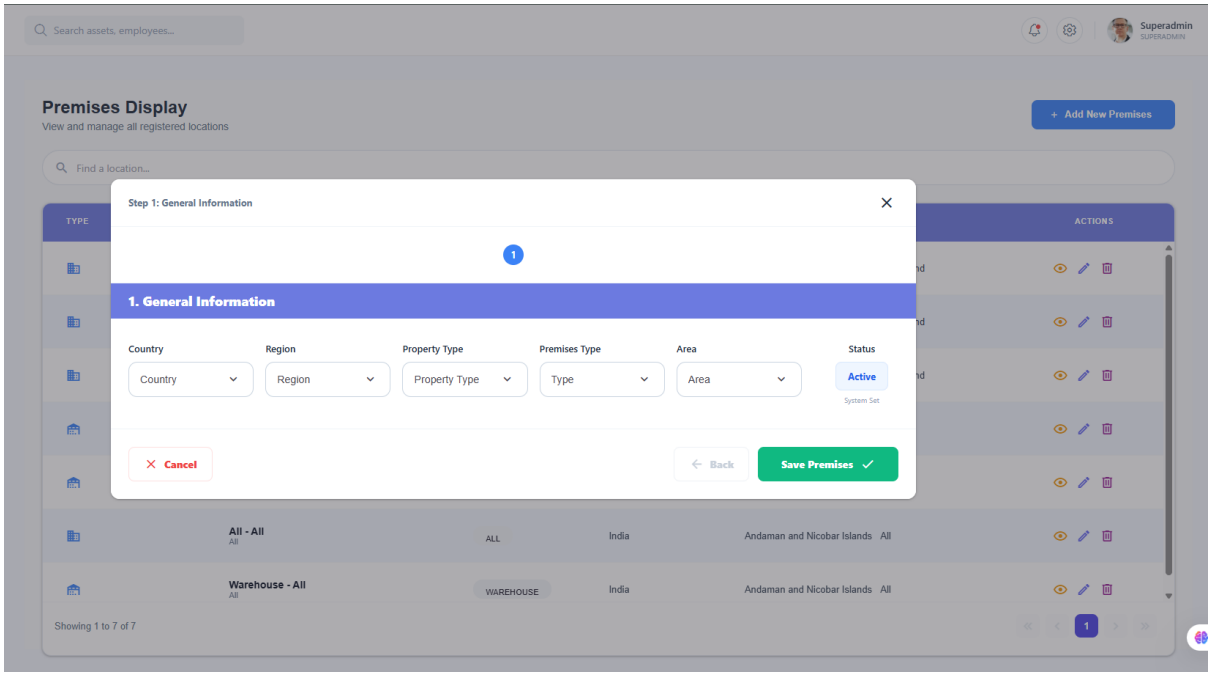
1. User opens **Submodule screen**
2. Selects module (Premises / Vehicles)
3. Applies filters (Country, Area, Type)
4. Clicks **Save**
5. Frontend calls API
6. Controller validates and stores data
7. Data saved in company_modules table
8. System applies this configuration only when conditions match

---

6. Purpose

- Customize modules based on location or type
- Enable flexible system behavior
- Avoid one fixed setup for all companies
- Support multiple configurations under one module

6.Premises Display (Master Screen) – Technical Structure



1. Frontend (UI Layer)

File Path:
apps/web/src/screens/PremisesMasterScreen.js

Purpose:
This screen is the **main list view for all premises (properties/locations)**.

Used to:

- View all premises (warehouses, offices, land, etc.)
- Filter by location (Country, Region, Area)
- Add new premises using **+ Add New**
- Perform actions:
    - View details
    - Edit premises
    - Delete premises

---

2. Backend (API Layer)

Controller

File:
server/controllers/officeController.js

Purpose:

- Fetch premises list
- Handle create, update, and delete operations
- Manage premises data based on company

Routes

File:
server/routes/office.js

Purpose:

- Defines API endpoints:
    - GET /api/offices → Get premises list
    - POST /api/offices → Add new premises
    - PUT /api/offices/:id → Update premises
    - DELETE /api/offices/:id → Delete premises
- Connects frontend → controller

---

3. Database (Schema Layer)

Main Table

Table Name: office_premises

Stores:

- premises_name
- premise_type (Owned / Rental)
- building_name
- Location details (country, region, area, address)
- Coordinates (if applicable)

Related Tables

1. office_owned_details

- Stores details for owned properties
- Example:

- Warranty
- Taxes
- Other ownership-related data

---

2. office_rental_details

- Stores details for rental properties
- Example:
  - Rent amount
  - Payment schedule
  - Lease deadlines

---

4. Who Can Access This Screen

✅ **Company Admin**

- Full access (Create, View, Edit, Delete)
- Manages all premises and related data

✅ **Super Admin**

- Full access across all companies
- Can view and manage all premises

---

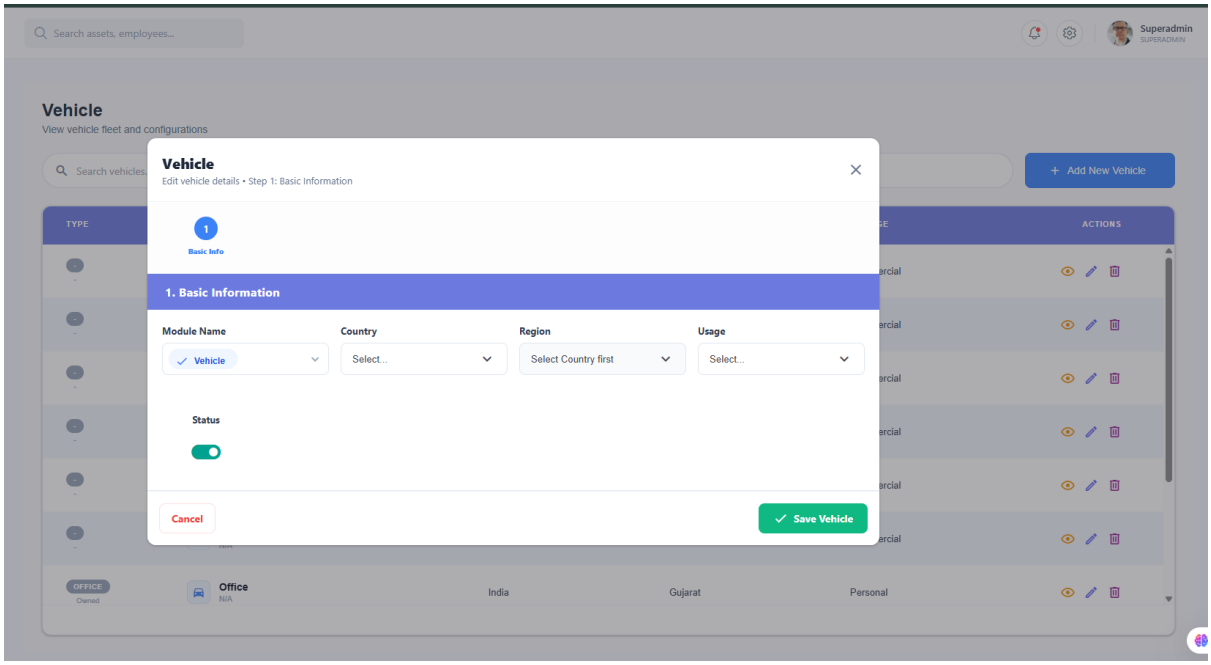
5. System Flow (Simple)

1. User opens **Premises screen**
2. System loads all premises from database
3. User can filter by location
4. User can:
   - Add new premises
   - Edit existing premises
   - Delete premises
5. Actions are sent to backend via API
6. Database updates accordingly
7. Updated data is shown in the list

6. Purpose

- Manage all company locations in one place
- Track owned and rental properties
- Store location and financial details
- Provide easy access and control over premises data

7. Vehicle – Technical Structure



1. Frontend (UI Layer)

File Path:
apps/web/src/screens/VehicleDisplayScreen.js

Purpose:
This modal is used to **add or edit a vehicle** in the fleet.

It works as a **step-by-step wizard**, starting with basic information.

Used to:

- Define vehicle setup before adding detailed info
- Set location and usage
- Control whether the vehicle is active

2. Modal Fields (

Step Type: Wizard Layout

- Step 1 focuses on **basic setup**
- Next steps will include detailed information (plate number, model, etc.)

Fields Explanation

- **Module Name**
  - Fixed as *Vehicle*
  - Indicates this record belongs to the vehicle module
- **Country & Region**
  - Defines where the vehicle operates
  - Helps in filtering and tracking
- **Usage**
  - Defines purpose of vehicle
  - Example:
    - Commercial
    - Personal
- **Status**
  - Active / Inactive
  - Controls visibility in the system

Save Action

- Clicking **Save**:
  - Creates or updates vehicle record
  - Stores data in database
  - Makes vehicle available in fleet list

---

3. Backend (API Layer)

Controller

File:
server/controllers/vehiclesController.js

Purpose:

- Handles:
    - Create vehicle
    - Update vehicle
    - Fetch vehicle list
    - Delete vehicle
- Validates and processes vehicle data

---

4. Database (Schema Layer)

Main Table

Table Name: vehicles

Stores:

- vehicle_id
- company_id
- country_id
- region_id
- usage (Commercial / Personal)
- status
- Additional details (in later steps)

---

5. System Flow (Simple)

1. User clicks **Add Vehicle**
2. Modal opens (Step 1: Basic Information)
3. User selects location and usage
4. User sets status
5. User clicks **Save**
6. Frontend calls API
7. Controller validates and saves data
8. Data stored in vehicles table
9. Vehicle appears in fleet list

---

6. Who Can Use This

✅ **Company Admin**

- Can add, edit, and manage vehicles

✅ **Super Admin**

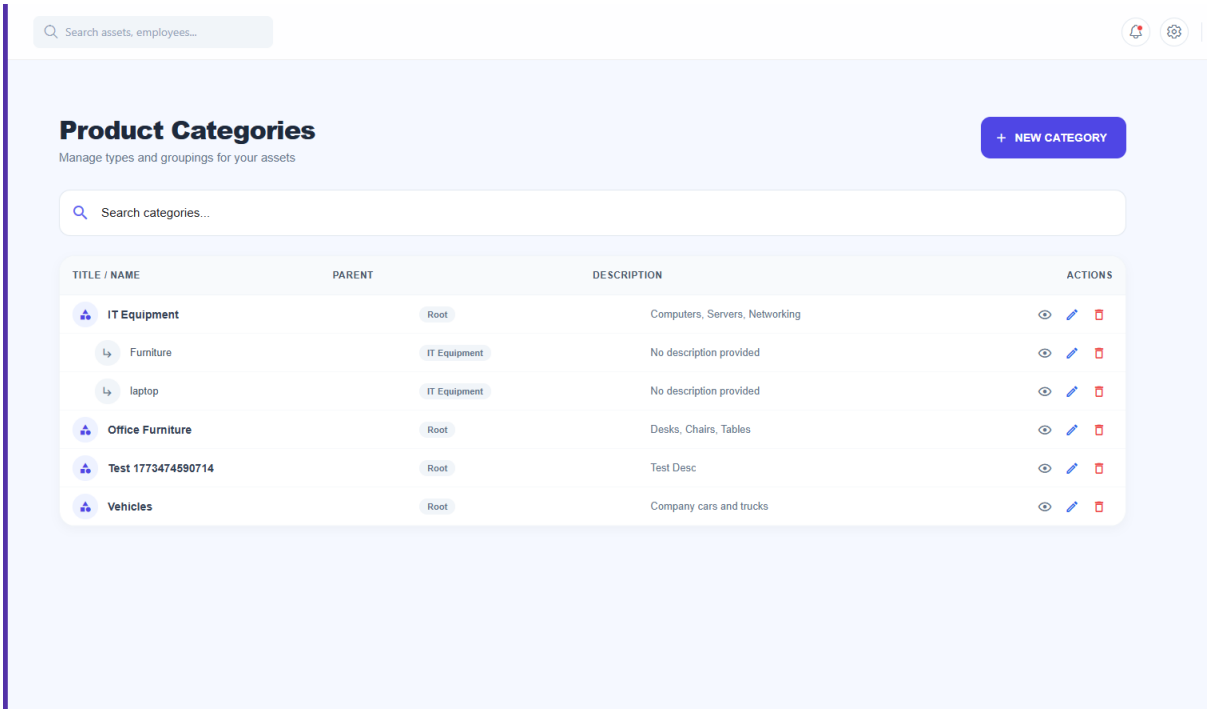
- Full access across all companies

7. Purpose

- Add and manage company vehicles
- Track vehicles by location and usage
- Control active/inactive fleet
- Prepare data for further details (next steps)

8.Product Categories (Asset Categories) – Technical Structure

---



1. Overview

This screen is used to **organize assets into categories and subcategories**.

It supports a **hierarchical structure (Parent → Child)** to keep assets well organized.

---

2. Main Feature – Hierarchical Grouping

- Categories can be created at **root level**

- Example: *IT Equipment*, *Office Furniture*

- Subcategories can be created under a parent
    - Example:
        - IT Equipment
          → Laptops
          → Accessories

👉 This helps in:

- Better organization
- Easy filtering
- Structured asset management

---

3. Frontend (UI Layer)

File Path:
apps/web/src/screens/AssetCategoriesScreen.js

Purpose:

- Displays category list in **tree structure**
- Shows parent and child relationships

Actions:

- Add new category
- Create subcategory under parent
- Edit category
- Delete category

---

4. Backend (API Layer)

Controller

File:
server/controllers/assetCategoriesController.js

Purpose:

- Handles:
    - Create category
    - Fetch categories (with hierarchy)
    - Update category

- Delete category
- Builds parent-child structure

Routes

File:
server/routes/categories.js

Purpose:

- Defines API endpoints:
  - GET /api/categories → Get all categories
  - POST /api/categories → Create category
  - PUT /api/categories/:id → Update category
  - DELETE /api/categories/:id → Delete category

---

5. Database (Schema Layer)

Main Table

Table Name: asset_categories

Stores:

- id
- company_id
- name
- description
- parent_id (links to another category)

---

6. How Hierarchy Works

- If parent_id = NULL → Root category
- If parent_id = some_id → Child category

👉 Example:

id	name	parent_id
1	IT Equipment	NULL
2	Laptops	1

---

7. System Flow (Simple)

1. User opens **Categories screen**
2. Clicks **+ New Category**
3. Enters category name
4. (Optional) selects parent category
5. Clicks **Save**
6. Data sent to backend
7. Stored in asset_categories table
8. Displayed in tree structure

---

8. Who Can Use This

✅ **Company Admin**

- Full access (create, edit, delete categories)

✅ **Super Admin**

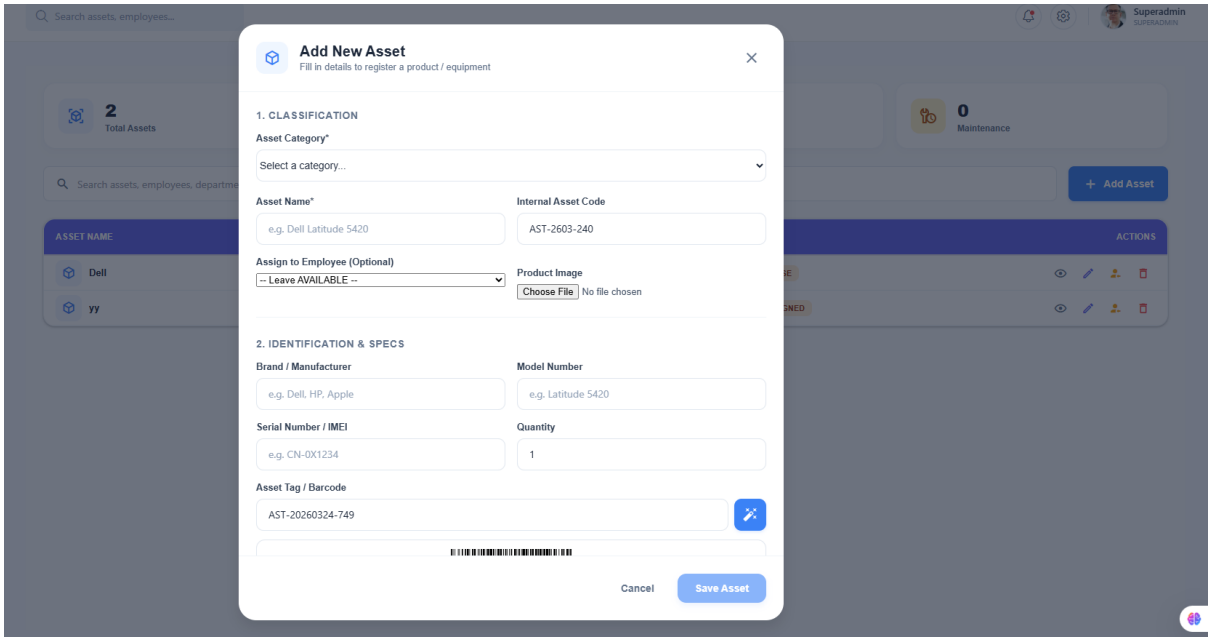
- Full access across all companies

---

9. Purpose

- Organize assets in structured way
- Enable filtering and reporting
- Support scalable inventory system
- Improve clarity in asset management

9.Asset Registry – Technical Structure



1. Overview

This modal is used to **register a new asset** in the system and optionally assign it to an employee.

---

2. Main Features

1. Classification Section

- **Asset Category**
    - Dropdown linked to category list
    - Helps organize the asset
- **Assign to Employee (Optional)**
    - Select employee to assign asset
    - If selected → asset status becomes **Assigned**
- **Internal Asset Code**
    - Unique identifier for each asset

---

2. Identification & Specs Section

- **Asset Tag / Barcode**
    - Auto-generated barcode
    - Can be scanned for quick identification

---

3. Frontend (UI Layer)

File Path:
apps/web/src/screens/AssetsScreen.js

Purpose:

- Displays asset list
- Opens **Add Asset modal**
- Handles asset creation UI

---

4. Backend (API Layer)

Controller

File:
server/controllers/assets.js

Purpose:

- Handles:
    - Create asset
    - Fetch asset list
    - Update asset
    - Delete asset
- Validates and processes asset data

Routes

File:
server/routes/assets.js

Purpose:

- Defines API endpoints:
    - GET /api/assets → Get assets
    - POST /api/assets → Create asset
    - PUT /api/assets/:id → Update asset
    - DELETE /api/assets/:id → Delete asset

---

5. Database (Schema Layer)

Main Table

Table Name: assets

Stores:

- asset_id
- company_id
- asset_code
- category_id
- status (Available / Assigned / etc.)
- quantity
- serial_number
- assigned_to (employee reference)

---

6. System Logic (Simple)

- If **assigned during creation** → status = **Assigned**
- If **not assigned** → status = **Available**

---

7. System Flow (Simple)

1. User opens **Assets screen**
2. Clicks **Add New Asset**
3. Enters asset details
4. Selects category
5. (Optional) assigns to employee
6. Clicks **Save**
7. Frontend calls API
8. Controller validates and saves data
9. Asset stored in assets table
10. Asset appears in list

---

8. Who Can Use This

✅ **Company Admin**

- Can create, edit, and manage assets

✅ **Super Admin**

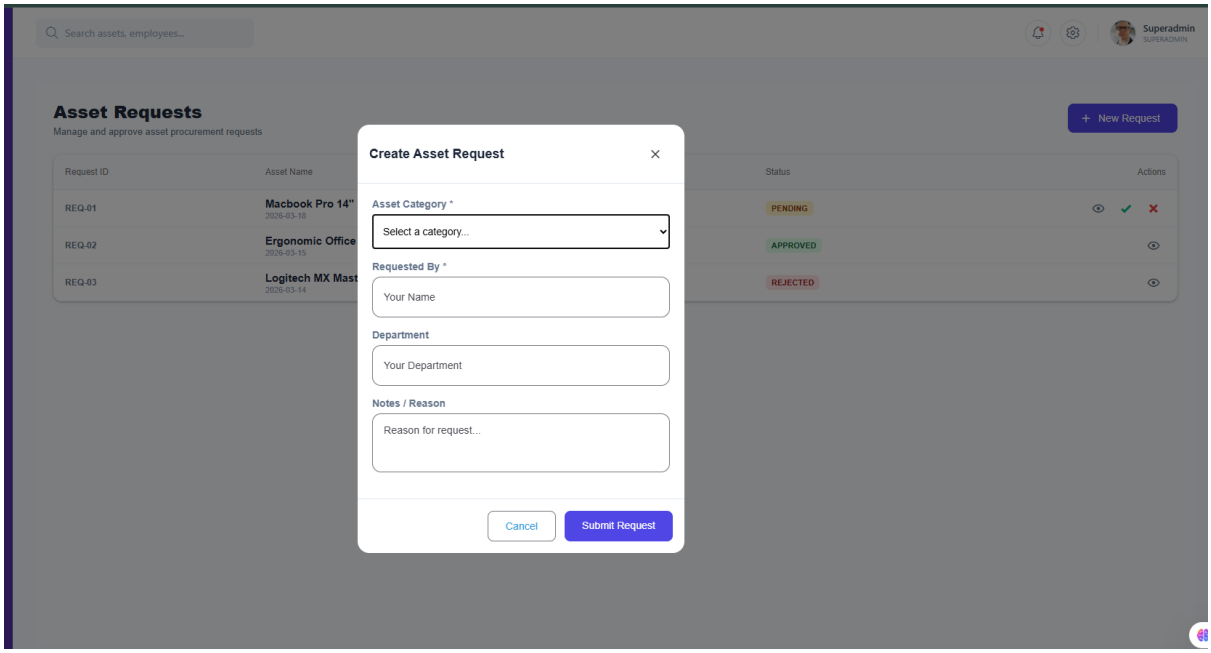
- Full access across all companies

---

9. Purpose

- Register new assets
- Track asset ownership and usage
- Assign assets to employees
- Enable barcode-based tracking

10.Asset Requests – Technical Structure



1. Overview

This screen is used to **manage asset requests**.

- Employees can request assets
- Admins can approve or reject requests

---

2. Main Features

1. Requests Table (Main Screen)

- Displays all requests
- Shows:

    - Asset / Category
    - Requested by (Employee)
    - Status (Pending, Approved, Rejected)
- **Quick Actions:**
    - ✅ Approve
    - ❌ Reject

👉 Status updates instantly after action

---

2. Create Request Modal

- Used to create a new request

Fields:

- **Asset Category** → Select required item
- **Reason** → Why the asset is needed

👉 Submits request for approval

---

3. Frontend (UI Layer)

File Path:
apps/web/src/screens/AssetRequestScreen.js

Purpose:

- Displays request list
- Opens request modal
- Handles approve/reject actions

---

4. Backend (API Layer)

Controller

File:
server/controllers/assets.js
(or requestsController.js if separated)

Purpose:

- Create request
- Fetch request list
- Approve request

- Reject request

Routes

File:
server/routes/requests.js

Purpose:

- Defines API endpoints:
    - GET /api/requests → Get all requests
    - POST /api/requests → Create request
    - PUT /api/requests/:id → Update status

---

5. Database (Schema Layer)

Main Table

Table Name: asset_requests

Stores:

- request_id
- company_id
- employee_id
- category_id
- reason
- status (Pending / Approved / Rejected)

---

6. System Logic (Simple)

- Default status → **Pending**
- If approved → status = **Approved**
- If rejected → status = **Rejected**

👉 On approval:

- Asset can be assigned to employee

---

7. System Flow (Simple)

1. Employee opens **Request screen**

2. Clicks **Create Request**
3. Selects category + enters reason
4. Submits request
5. Request stored with status **Pending**
6. Admin reviews request
7. Admin clicks Approve / Reject
8. Status updates
9. If approved → asset assignment can be done

---

8. Who Can Use This

✅ **Employees**

- Create requests

✅ **Company Admin**

- Approve / Reject requests

✅ **Super Admin**

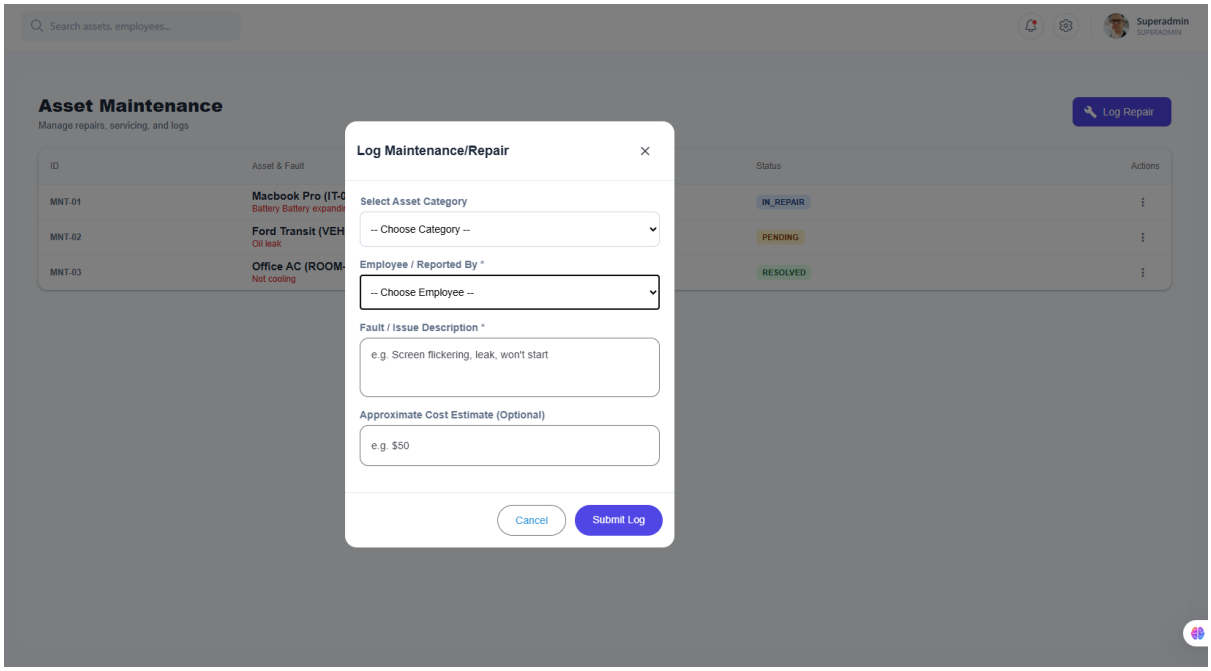
- Full access

---

9. Purpose

- Manage asset demand
- Control approvals
- Track request history
- Ensure proper allocation

11.Asset Maintenance – Technical Structure



1. Overview

This modal is used to **create a maintenance or repair ticket** for an asset.

- Records issues
- Tracks repair details
- Helps manage asset condition

---

2. Main Features

1. Target Selection

- **Asset Category**
    - Filters and selects the relevant asset
- **Reported By (Employee)**
    - Identifies who reported the issue
    - Helps track responsibility

---

2. Issue Details

- **Fault / Issue Description**
    - Describes the problem

    - Example:
        - “Won’t start”
        - “Screen flickering”
- **Approximate Cost (Optional)**
    - Estimated repair cost
    - Helps in budgeting

---

3. Frontend (UI Layer)

File Path:
apps/web/src/screens/AssetMaintenanceScreen.js

Purpose:

- Opens repair modal
- Captures issue details
- Submits maintenance ticket

---

4. Backend (API Layer)

Controller

File:
server/controllers/assets.js

Purpose:

- Create maintenance ticket
- Validate input data
- Save repair details

Routes

File:
server/routes/maintenance.js

Purpose:

- Defines API endpoints:
    - POST /api/maintenance → Create ticket
    - GET /api/maintenance → Fetch tickets
    - PUT /api/maintenance/:id → Update status

---

5. Database (Schema Layer)

Main Table

Table Name: maintenance_tickets

Stores:

- ticket_id
- company_id
- asset_id
- reported_by (employee_id)
- issue_description
- estimated_cost
- status (Open / In Repair / Resolved)

---

6. System Logic (Simple)

- Ticket is created with status = **Open**
- When work starts → status = **In Repair**
- When completed → status = **Resolved**

👉 Asset behavior:

- **In Repair** → cannot be assigned
- **Resolved** → available again

---

7. System Flow (Simple)

1. User opens **Maintenance screen**
2. Clicks **Log Repair**
3. Selects asset and reporter
4. Enters issue description
5. Adds estimated cost (optional)
6. Clicks **Save**
7. Frontend calls API
8. Controller validates and stores data
9. Ticket saved in maintenance_tickets

10. Appears in maintenance list

---

8. Who Can Use This

✅ **Company Admin**

- Create and manage repair tickets

✅ **Super Admin**

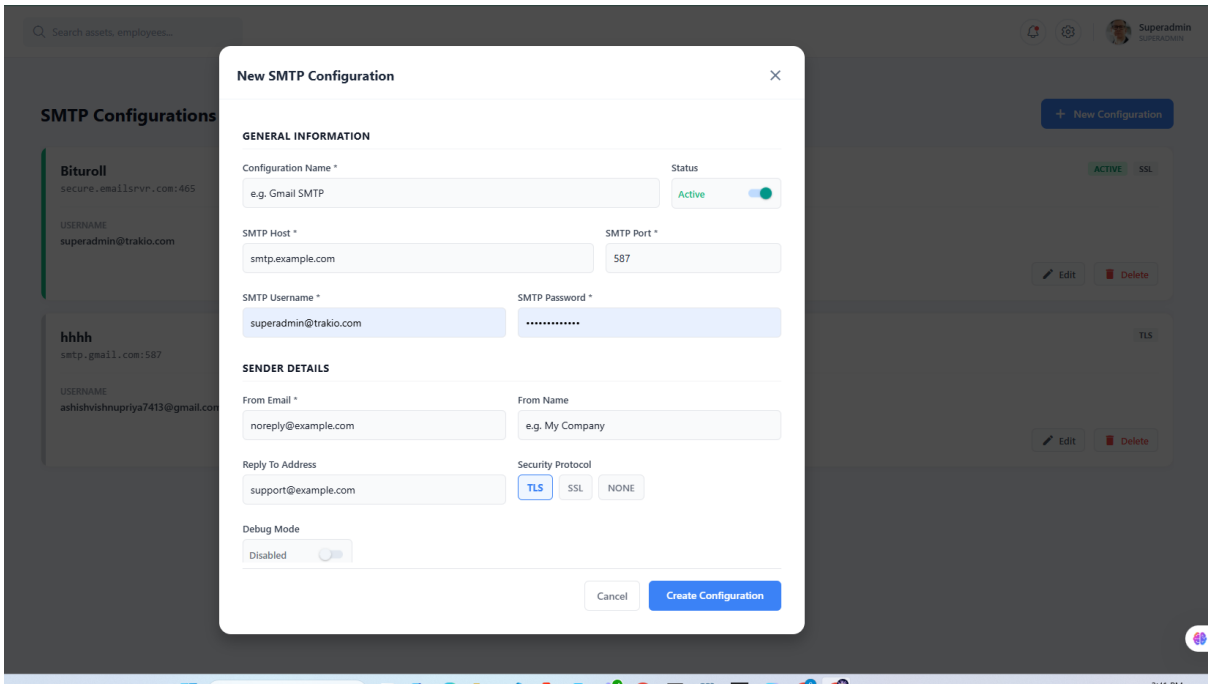
- Full access

---

9. Purpose

- Record asset issues
- Manage repair workflow
- Track maintenance cost
- Maintain asset reliability

12.SMTP Configurations – Technical Structure



1. Overview

This screen is used to **configure email server settings** for a company.

It enables the system to send:

- Notifications
- Alerts
- Request updates
- Maintenance updates

---

2. Main Features

1. Configuration List (Cards/Table)

Displays all configured email servers.

Shows:

- **Host & Port**
    - Example: smtp.gmail.com:587
- **From Email**
    - Email visible to users
    - Example: info@company.com
- **Encryption Type**
    - SSL or TLS
- **Status**
    - Active / Inactive

---

2. Actions

- **Edit** → Update configuration
- **Delete** → Remove configuration

---

3. Frontend (UI Layer)

File Path:
apps/web/src/screens/settings/SmtpScreen.js

Purpose:

- Display SMTP configurations
- Add / edit / delete settings
- Manage active configuration

---

4. Backend (API Layer)

Controller

File:
server/controllers/smtpController.js

Purpose:

- Save SMTP settings
- Update configurations
- Delete configurations
- Handle email setup logic

Routes

File:
server/routes/smtp.js

Purpose:

- Defines API endpoints:
    - GET /api/smtp → Get configurations
    - POST /api/smtp → Add configuration
    - PUT /api/smtp/:id → Update configuration
    - DELETE /api/smtp/:id → Delete configuration

---

5. Database (Schema Layer)

Main Table

Table Name: company_smtp_settings

Stores:

- id
- company_id
- host
- port
- username
- password (encrypted)
- encryption (SSL / TLS)
- from_email

- status

---

6. System Logic (Simple)

- Only **active SMTP configuration** is used for sending emails
- Password is stored securely (encrypted)
- System uses this configuration for all outgoing emails

---

7. System Flow (Simple)

1. User opens **SMTP Settings**
2. Adds or edits configuration
3. Enters host, port, email, and credentials
4. Selects encryption type
5. Sets status as Active
6. Clicks **Save**
7. Data sent to backend
8. Stored in company_smtp_settings
9. System uses this config for sending emails

---

8. Who Can Use This

✅ **Company Admin**

- Configure email settings

✅ **Super Admin**

- Full access

❌ **Employees / Staff**

- No access

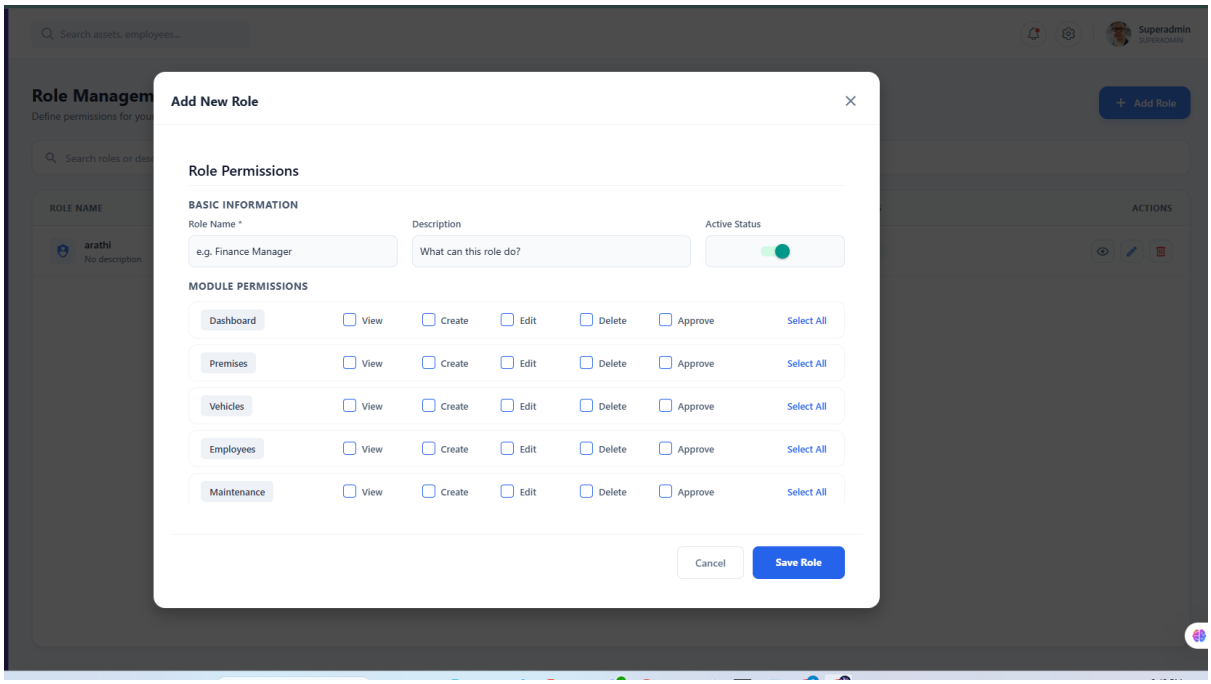
---

9. Purpose

- Enable system email communication
- Send alerts and notifications

- Support automated workflows
- Maintain centralized email configuration

13.Add New Role & Permissions – Technical Structure



1. Overview

This modal is used to **create roles and define permissions** for users.

It helps control **who can access what in the system**.

---

2. Main Features

1. Basic Information

- **Role Name**
    - Example: *Finance Manager*, *Technician*
- **Description**
    - Explains the role purpose

---

2. Module Permissions (Access Matrix)

This section defines access for each module.

Modules (Rows)

- Dashboard
- Premises
- Vehicles
- Employees
- Maintenance

Permissions (Columns)

- **View** → Can see the page
- **Create** → Can add new records
- **Edit** → Can update records
- **Delete** → Can remove records
- **Approve** → Can approve actions

Select All

- Enables all permissions for that module

---

3. Frontend (UI Layer)

File Path:
apps/web/src/screens/RolesScreen.js

Purpose:

- Display roles list
- Open role creation modal
- Manage permission matrix

---

4. Backend (API Layer)

Controller

File:
server/controllers/roles.js

Purpose:

- Create role
- Save permissions

- Fetch roles
- Update role and permissions
- Delete role

Routes

File:
server/routes/roles.js

Purpose:

- Defines API endpoints:
    - GET /api/roles → Get roles
    - POST /api/roles → Create role
    - PUT /api/roles/:id → Update role
    - DELETE /api/roles/:id → Delete role

---

5. Database (Schema Layer)

Main Tables

1. roles

Stores:

- id
- company_id
- role_name
- description

---

2. role_permissions

Stores:

- role_id
- module_name
- view_access
- create_access
- edit_access
- delete_access

- approve_access

---

6. System Logic (Simple)

- Each user is assigned a **role**
- Each role has **permissions per module**
- System checks permissions before allowing actions

---

7. System Flow (Simple)

1. Admin opens **Roles screen**
2. Clicks **Add New Role**
3. Enters role name and description
4. Selects permissions for each module
5. Clicks **Save**
6. Frontend sends data to API
7. Controller saves role + permissions
8. Data stored in roles and role_permissions
9. Role is available for assigning to users

---

8. Who Can Use This

✅ **Company Admin**

- Create and manage roles

✅ **Super Admin**

- Full access

---

9. Purpose

- Control system access
- Define responsibilities
- Improve security
- Separate duties between roles